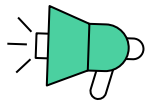


Репликация и масштабирование

Часть 2



Проверка связи



Если у вас нет звука:

- убедитесь, что на вашем устройстве и на колонках включён звук
- обновите страницу вебинара (или закройте страницу и заново присоединитесь к вебинару)
- откройте вебинар в другом браузере
- перезагрузите компьютер (ноутбук) и заново попытайтесь зайти



Поставьте в чат:

- + если меня видно и слышно
- если нет

Рекомендации



Если смотрите с компьютера

- Используйте браузеры **Google Chrome** или **Microsoft Edge**
- Если есть проблемы с изображением или звуком, обновите страницу — **F5**



Если смотрите с мобильного телефона или планшета

- Перейдите с мобильного интернет-соединения на **Wi-Fi**
- Если есть проблемы с изображением или звуком, перезапустите приложение

Юрий Новиков

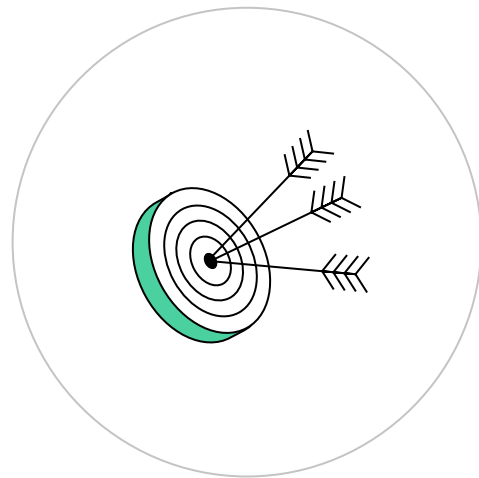
О спикере:

- В IT с 2000 года.
- В настоящее время ведущий разработчик в ОМК-ИТ, ранее занимал руководящие должности.
- Принимал участие в разработке веб-приложений и прикладного ПО. Одна из систем - система мониторинга городского пассажирского транспорта.
- Программирую: Delphi, C#, Php, JavaScript.
- Проектирует базы, взаимодействию с базами посредством СУБД MySql, MSQl, Oracle.
- Проводит курсы по реляционным базам данных.



Цели занятия

- Поговорим о масштабировании баз данных
- Разберем, какие методы и технологии партиционирования и шардинга бывают, как они работают

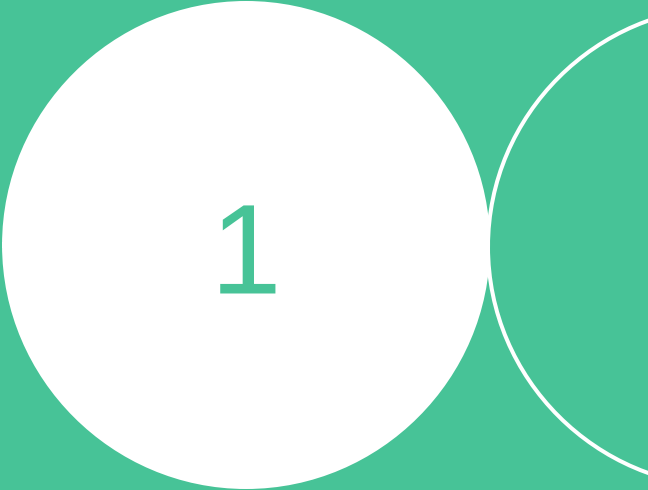


План занятия

- 1 Масштабирование
- 2 Вертикальное масштабирование (scaling up)
- 3 Горизонтальное масштабирование (scaling out)
- 4 Шардинг (shard—сегментировать). Вертикальный и горизонтальный
- 5 Примеры реализации в docker



Масштабирование



1



Масштабирование — это оптимизация производительности системы при изменении поступающей нагрузки

Типы масштабирования

1

Scaling up

масштабирование
вертикальное

2

Scaling out

масштабирование
горизонтальное

3

Scaling back

часть данных открыты,
часть данных
заархивированы

Вертикальное масштабирование

Scaling up



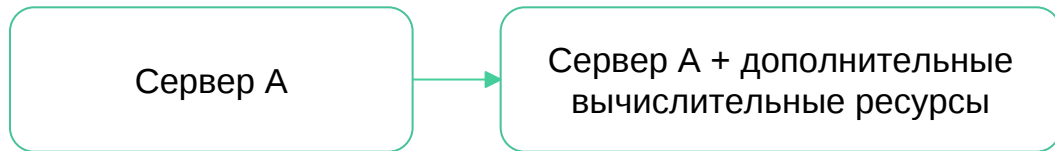
2



Вертикальное масштабирование предполагает наращивание мощностей сервера

Основным преимуществом метода является его простота.

Нет необходимости переписывать код при добавлении мощностей, а управлять одним крупным сервером намного проще, чем целой системой



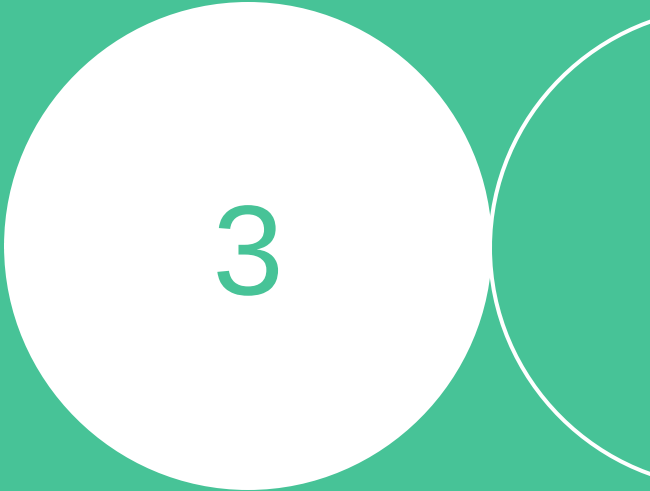
Недостатки вертикального масштабирования

- Масштабирование ресурсов одного сервера имеет вполне конкретные аппаратные ограничения
- Сервер с кратным объёмом вычислительных ресурсов в большинстве случаев оказывается дороже, чем несколько менее мощных серверов, дающих в сумме такую же производительность



Горизонтальное масштабирование

Scaling out



3



Горизонтальное масштабирование означает увеличение производительности за счёт разделения данных

Такой способ предполагает увеличение производительности без снижения отказоустойчивости

Основные виды горизонтального масштабирования

1

Репликация (master-slave, master-master)

2

Партицирование

3

Шардинг

Репликация

Это копирование данных между серверами



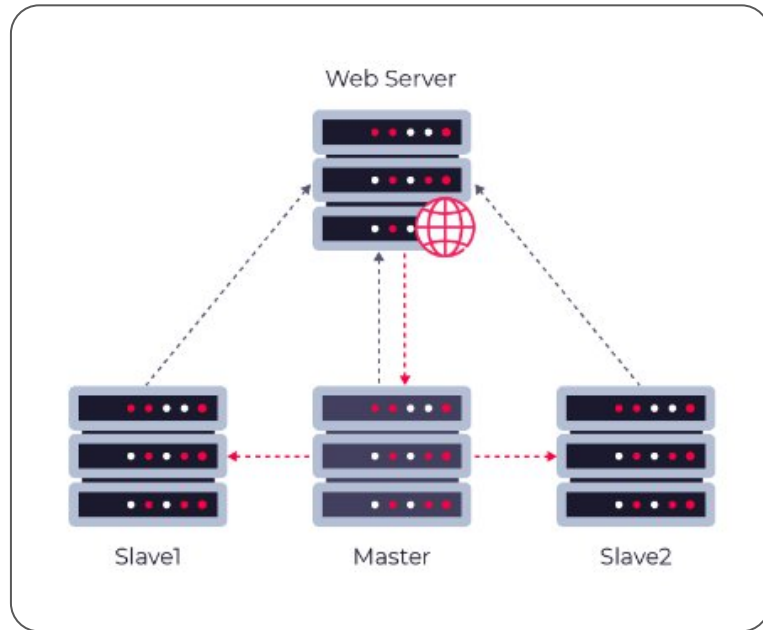
Преимущество:

Большое количество копий данных. Если головной сервер выходит из строя, любой другой сможет его заменить



Недостаток:

Рассинхронизация и задержки при передаче данных между серверами



Наиболее распространенный паттерн применения – отказоустойчивость

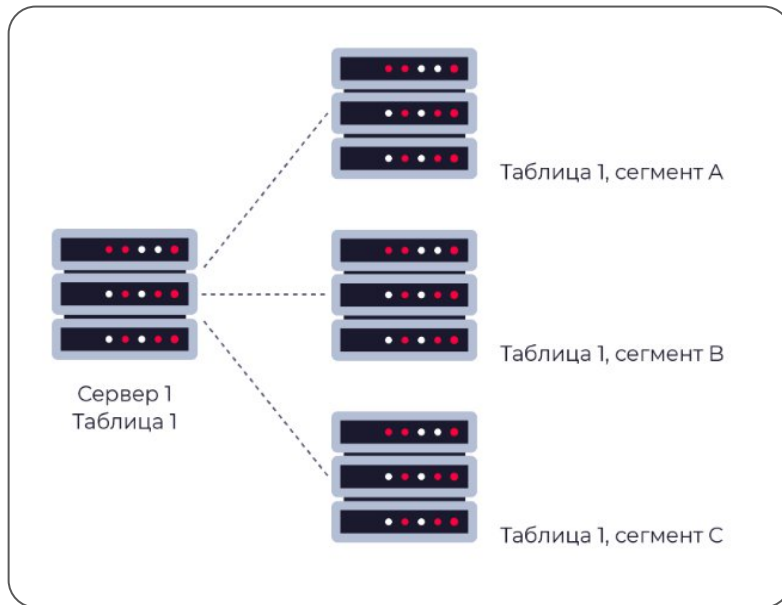
Партиционирование

Это разделение данных на части по какому-либо признаку

Дата	№ отчёта
15.01.2024	2142
29.01.2024	2143
14.02.2024	2144
18.02.2024	2145
28.02.2024	2146
15.03.2024	2147

* Разделение таблицы по признаку даты

[Источник](#)



Шардинг

Вертикальный и горизонтальный



4

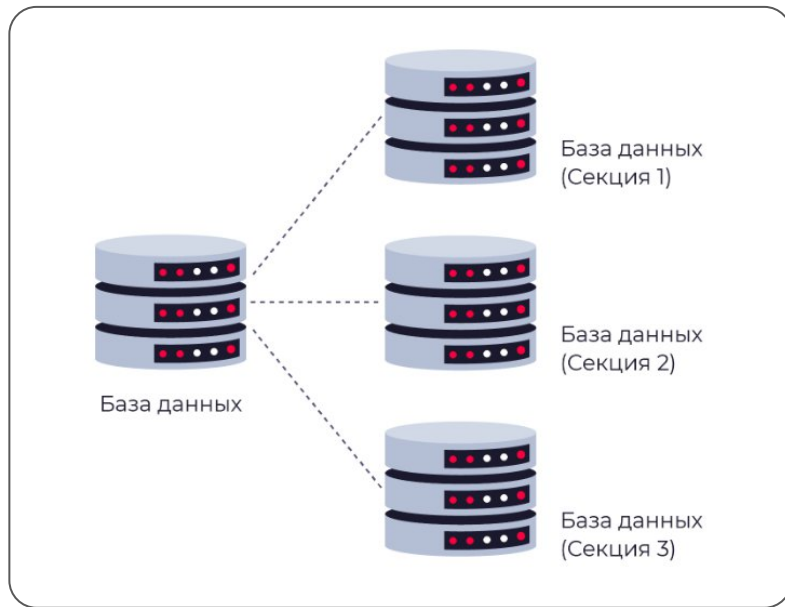
Шардинг

Это разделение данных БД и их
раздельное хранение на различных
физических серверах

Существует два вида шардинга:

- вертикальный
- горизонтальный

Шардинг является наиболее предпочтительным
для высоконагруженных сервисов. Но стоит
отметить, что это достаточно сложно реализовать,
так как необходимо учитывать межсерверное
взаимодействие



Вертикальный шардинг

Столбцы исходной таблицы выделяются в новые таблицы.
Каждая таблица содержит одинаковое количество строк,
но меньшее количество столбцов

Например:

→

Существует таблица **users**
(id, user, password)

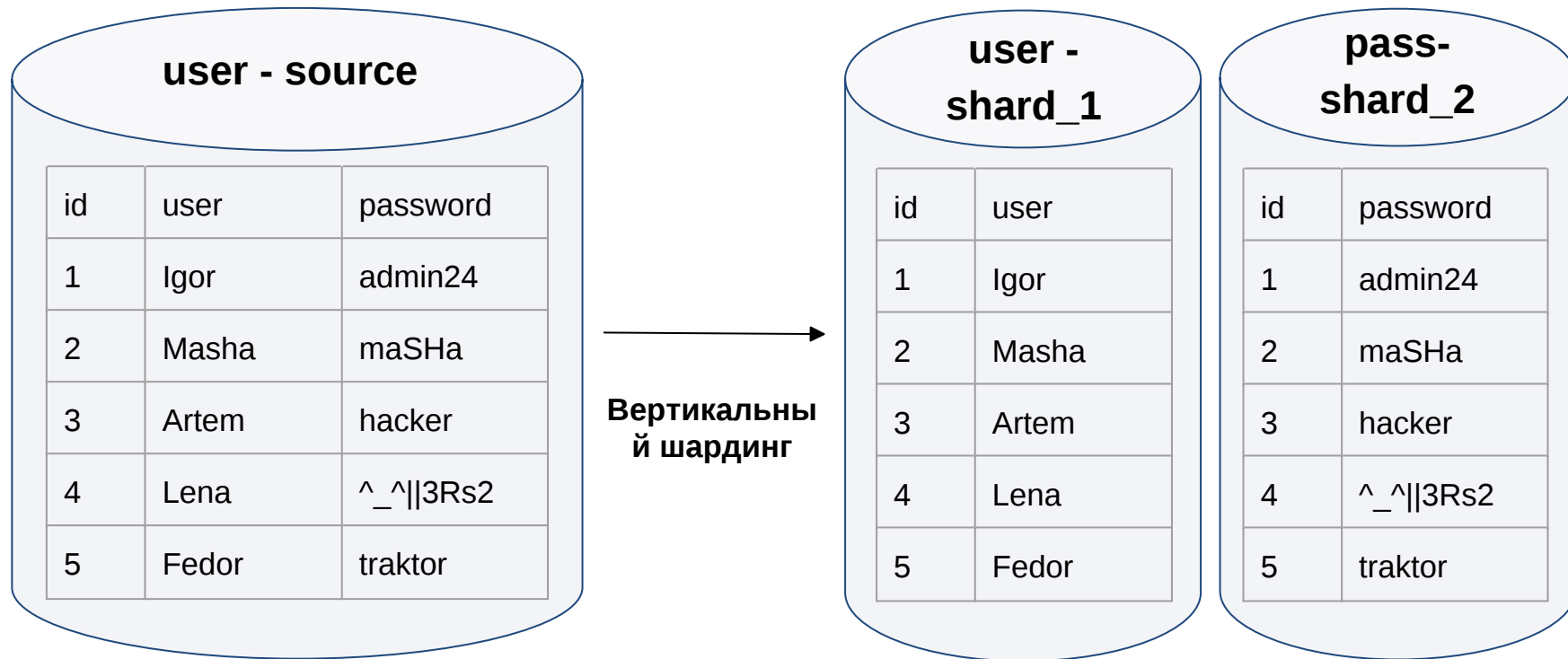
→

Разбиваем её на две
таблицы: **users** (id, user) и
password (id, password)

*

Таблицу **users** оставляем
на одном сервере, а
таблицу **password**
переводим на другой

Вертикальный шардинг



Горизонтальный шардинг

Разделение таблицы на несколько меньших таблиц. Каждая таблица представляет собой отдельное хранилище данных и содержит такое же количество столбцов, но меньшее количество строк

Например:



Существует таблица **users** (id, user, password), в которой хранится информация о пользователях с id в диапазоне от 0 до 1000000

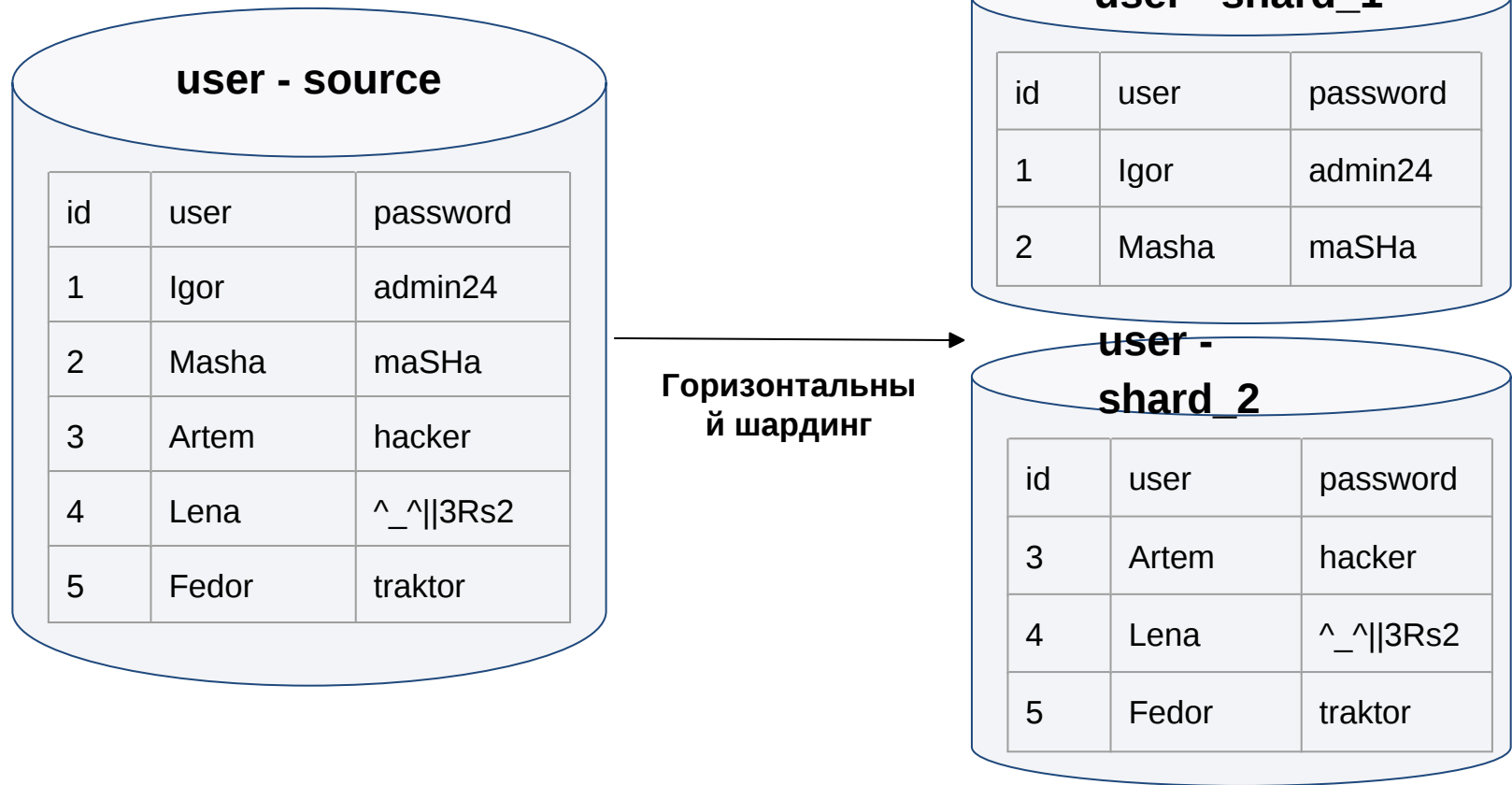


Разбиваем её на две одинаковые таблицы **users** (id, user, password)



- В одной хранится информация о пользователях с id в диапазоне от 0 до 500000
- Во второй хранится информация о пользователях с id в диапазоне от 500001 до 1000000

Горизонтальный шардинг



Примеры реализации в docker



5

Давайте вспомним

Вопрос: Что такое технология **docker**?



Давайте вспомним

Вопрос: Что такое технология **docker**?

Ответ: Это технология контейнеризации позволяющая изолированно разворачивать приложения



Давайте вспомним

Вопрос: Что такое **docker-образ (image)**?



Давайте вспомним

Вопрос: Что такое **docker-образ (image)**?

Ответ: Это неизменяемый файл, содержащий исходный код, библиотеки, зависимости, инструменты и другие файлы, необходимые для запуска приложения



Давайте вспомним

Вопрос: Что такое **docker-контейнер**?



Давайте вспомним

Вопрос: Что такое **docker-контейнер**?

Ответ: Это виртуализированная среда выполнения, в которой пользователи могут изолировать приложения от хостовой системы

Эти контейнеры представляют собой компактные портативные хосты, в которых можно быстро и легко запустить приложение



Партиционирование

Для запуска postgresQL используем docker:

```
docker run --name neto-pg -e POSTGRES_PASSWORD=netology -d postgres
```

Для запуска SQL скриптов можно использовать:

- интерактивный терминал psql (входит в пакет postgresQL)
- графический клиент [dbeaver](#)

Для входа в терминал psql можно воспользоваться командой:

```
docker exec -it neto-pg psql -U postgres
```

[Инструкция для подключения из dbeaver](#)

Партиционирование

Создадим отношение:

```
CREATE TABLE workers (  
  id uuid not null,  
  category int  
) PARTITION BY RANGE (category);
```

В отношении 2 атрибута: `id` и `category` партиционирование будем производить по признаку `category`

Партиционирование

Добавим 3 партии :

```
CREATE TABLE workers_1  
PARTITION OF workers  
FOR VALUES FROM (1) TO (4);
```

```
CREATE TABLE workers_2  
PARTITION OF workers  
FOR VALUES FROM (4) TO (7);
```

```
CREATE TABLE workers_3  
PARTITION OF workers  
FOR VALUES FROM (7) TO (10);
```

Деление производится по диапазону значений category

Партиционирование

Наполним отношение `workers` данными: включим расширение для использования UUID:

```
CREATE EXTENSION IF NOT EXISTS "uuid-osspl";
```

Заполним отношение значениями с категориями от 1 до 9:

```
insert into workers (id, category)
values (
    uuid_generate_v4(),
    generate_series(1::int, 9::int, 1)
);
```

Партиционирование

Выполним проверки:

```
select * from workers_1;
```

```
select * from workers_2;
```

```
select * from workers_3;
```

Каждая партиция содержит данные из определенной для неё категории

Давайте вспомним

Вопрос: Что такое технология **docker-compose**?



Давайте вспомним

Вопрос: Что такое технология **docker-compose**?

Ответ: Это инструмент для одновременного управления несколькими контейнерами



Давайте вспомним

Вопрос: За счёт чего можно объединить контейнеры в сеть?



Давайте вспомним

Вопрос: За счёт чего можно объединить контейнеры в сеть?

Ответ: `docker-network`



Давайте вспомним

Вопрос: Что такое **docker volume (том)**?



Давайте вспомним

Вопрос: Что такое **docker volume (том)**?

Ответ: Это внешнее хранилище данных docker-контейнера



Шардинг

Для запуска PostgreSQL используем docker compose. Создадим файл `docker-compose.yml` и добавим в него узел master:

```
version: '3.8'

services:
  postgres_b:
    image: postgres:latest
    container_name: "postgres_b"
    env_file: .env
    environment:
      PGDATA: "/var/lib/postgresql/data/pgdata"
    ports:
      - "5632:5432"
    volumes:
      - ./conf/postgres_b/shards.sql:/docker-entrypoint-initdb.d/start.sql
```

Основные настройки:

- `postgres_b` – имя контейнера
- `image: postgres:latest` – образ PostgreSQL
- `env_file` – путь к файлу переменных окружения
- `ports` – маппинг портов docker network на локальный хост
- `volume` – подключение внешнего хранилища
- `./conf/postgres_b/shards.sql:/docker-entrypoint-initdb.d/start.sql` – путь к скрипту инициализации

Шардинг

Добавим в `docker-compose.yml` шарды:

```
postgres_b1:  
  image: postgres:latest  
  container_name: "postgres_b1"  
  env_file: .env  
  environment:  
    PGDATA: "/var/lib/postgresql/data/pgdata"  
  ports:  
    - "5633:5432"  
  volumes:  
    - ./conf/postgres_b1/shards.sql:/docker-entrypoint-initdb.d/start.sql
```

Шардинг

Добавим в `docker-compose.yml` шарды:

```
postgres_b2:  
  image: postgres:latest  
  container_name: "postgres_b2"  
  env_file: .env  
  environment:  
    PGDATA: "/var/lib/postgresql/data/pgdata"  
  ports:  
    - "5634:5432"  
  volumes:  
    - ./conf/postgres_b2/shards.sql:/docker-entrypoint-initdb.d/start.sql
```

Шардинг

Добавим в `docker-compose.yml` шарды:

```
postgres_b3:  
  image: postgres:latest  
  container_name: "postgres_b3"  
  env_file: .env  
  environment:  
    PGDATA: "/var/lib/postgresql/data/pgdata"  
  ports:  
    - "5635:5432"  
  volumes:  
    - ./conf/postgres_b3/shards.sql:/docker-entrypoint-initdb.d/start.sql
```

Шардинг

Структура проекта для [docker-compose.yml](#):

```
▼ DOCKER-PG-SHARDS
  ▼ conf
    ▼ postgres_b
      shards.sql
    ▼ postgres_b1
      shards.sql
    ▼ postgres_b2
      shards.sql
    ▼ postgres_b3
      shards.sql
  .env
  docker-compose.yml
```

Шардинг

Файл `shards.sql`:

```
CREATE EXTENSION postgres_fdw;

/* SHARD 1 */
/* регистрируем сервер с шардом*/
CREATE SERVER books_1_server
  FOREIGN DATA WRAPPER postgres_fdw
  OPTIONS (host 'postgres_b1', port '5432', dbname 'books');

/* настраиваем маппинг*/
CREATE USER MAPPING FOR "postgres"
  SERVER books_1_server
  OPTIONS (user 'postgres', password 'postgres');

/* создаём таблицу*/
CREATE FOREIGN TABLE books_1
(
  id bigint not null,
  category_id int not null,
  author character varying not null,
  title character varying not null,
  year int not null
) SERVER books_1_server
  OPTIONS (schema_name 'public', table_name 'books');
```

Шардинг

Файл `shards.sql`:

`SHARD 2` и `SHARD 3` аналогично изменяются строки:

```
CREATE SERVER books_1_server на CREATE SERVER books_2_server и CREATE SERVER books_3_server
OPTIONS (host 'postgres_b1', port '5432', dbname 'books'); на OPTIONS (host 'postgres_b2', port '5432', dbname 'books'); и OPTIONS (host 'postgres_b3', port
'5432', dbname 'books');
CREATE FOREIGN TABLE books_1 на CREATE FOREIGN TABLE books_2 и CREATE FOREIGN TABLE books_3
```

Шардинг

Файл `shards.sql`:

```
/* добавляем представления */  
CREATE VIEW books AS  
SELECT *  
FROM books_1  
UNION ALL  
SELECT *  
FROM books_2  
UNION ALL  
SELECT *  
FROM books_3;
```

Шардинг

Файл `shards.sql`:

```
/* добавляем критерии заполнения */
CREATE RULE books_insert AS ON INSERT TO books
DO INSTEAD NOTHING;
CREATE RULE books_update AS ON UPDATE TO books
DO INSTEAD NOTHING;
CREATE RULE books_delete AS ON DELETE TO books
DO INSTEAD NOTHING;

CREATE RULE books_insert_to_1 AS ON INSERT TO books
WHERE (category_id <= 3)
DO INSTEAD INSERT INTO books_1
VALUES (NEW.*);

CREATE RULE books_insert_to_2 AS ON INSERT TO books
WHERE (category_id > 3 and category_id <= 6)
DO INSTEAD INSERT INTO books_2
VALUES (NEW.*);

CREATE RULE books_insert_to_3 AS ON INSERT TO books
WHERE (category_id > 6)
DO INSTEAD INSERT INTO books_3
VALUES (NEW.*);
```

Шардинг

Файл `postgres_b1/shards.sql`:

```
/* создаём отношение */
CREATE TABLE books
(
    id bigint not null,
    category_id int not null,
    CONSTRAINT category_id_check CHECK (category_id <= 3),
    author character varying not null,
    title character varying not null,
    year int not null
);
/* добавляем индексацию по полю category_id*/
CREATE INDEX books_category_id_idx ON books USING btree(category_id)
```

Шардинг

Файл [postgres_b2/shards.sql](#) :

```
CREATE TABLE books
(
  id bigint not null,
  category_id int not null,
  CONSTRAINT category_id_check CHECK (category_id > 3 and category_id <= 6),
  author character varying not null,
  title character varying not null,
  year int not null
);

CREATE INDEX books_category_id_idx ON books USING btree(category_id)
```

Шардинг

Файл [postgres_b3/shards.sql](#) :

```
CREATE TABLE books
(
  id bigint not null,
  category_id int not null,
  CONSTRAINT category_id_check CHECK (category_id > 6),
  author character varying not null,
  title character varying not null,
  year int not null
);

CREATE INDEX books_category_id_idx ON books USING btree(category_id)
```

Шардинг

Для запуска [docker-compose](#) файла воспользуемся командой:

```
docker-compose up -d (версия compose < 2)
```

```
docker compose up -d (версия compose >= 2)
```

Шардинг

Для заполнения таблицы выполним SQL запрос:

```
INSERT INTO books (id, category_id, author, title, year)
VALUES (1, 1, 'Author1', 'book1', 1980),
      (2, 2, 'Author2', 'book2', 1997),
      (3, 5, 'Author3', 'book3', 2005),
      (4, 2, 'Author4', 'book4', 2005),
      (5, 7, 'Author5', 'book5', 2005),
      (6, 5, 'Author6', 'book6', 2001),
      (7, 9, 'Author7', 'book7', 1917),
      (8, 2, 'Author8', 'book8', 1956),
      (9, 1, 'Author9', 'book9', 1972),
      (10, 8, 'Author10', 'book10', 1993);
```

Шардинг

Выполним проверки:

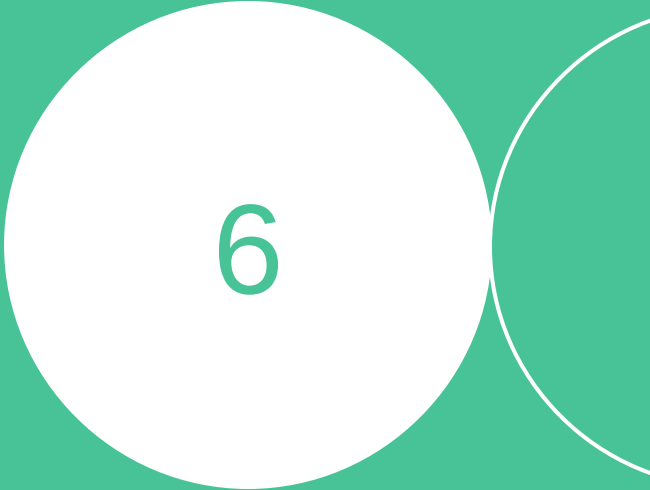
```
select * from books_1;
```

```
select * from books_2;
```

```
select * from books_3;
```

Каждый шард содержит данные из определенной для неё категории

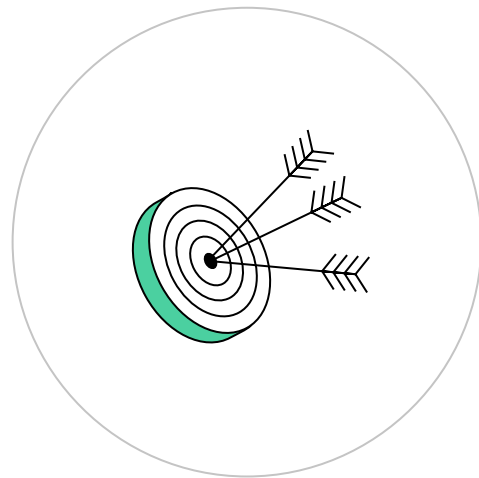
Итоги



6

Сегодня мы:

- Рассмотрели принципы масштабирования, шардинга
- Поговорили о вертикальном и горизонтальном масштабировании
- Настроили партиционирование и шардирование на примере postgresSQL



Домашнее задание

7

Домашнее задание

Давайте вместе посмотрим домашнее задание

- Вопросы по домашней работе задавайте в разделе "Вопросы по заданию"
- Задачи можно сдавать по частям
- Зачёт по домашней работе проставляется после того, как приняты все задачи

Репликация и масштабирование

Часть 2

